

Name: Shaik Mohammed Abrar

Reg no: 192472185

Course Name: Design and Analysis of Algorithm

Course Code: CSA0652

Assessment: CO5-Assessment Tool 2

Report 3

Q67: Approximation Maximum Cut Problem

1. Aim

The aim of this experiment is to design and implement an approximation algorithm for the **Maximum Cut Problem**. The objective is to partition the vertices of a graph into two disjoint sets such that the number of edges crossing between the two sets is maximized. Since the Maximum Cut Problem is NP-Hard, an approximation approach is used to obtain a near-optimal solution efficiently.

2. Problem Description

Given an undirected graph $G(V,E)$, the Maximum Cut Problem seeks a partition of the vertex set (V) into two subsets (A) and (B) such that the number of edges connecting vertices in different subsets is as large as possible.

For a graph: $G(V,E)$

A cut is defined as: (A, B)

Where:

$$A \cup B = V, A \cap B = \emptyset$$

The goal is to maximize the cut size, which is the number of edges crossing from set A to set B.

Because finding the exact maximum cut is computationally expensive for large graphs, approximation algorithms are used to obtain good solutions within reasonable time.

3. Objectives

1. Partition graph vertices into two subsets.
2. Maximize the number of crossing edges.
3. Implement an approximation algorithm.
4. Reduce computational complexity compared to exact algorithms.
5. Analyze the quality and efficiency of the solution.

4. Algorithm

Step 1: Input

Read:

1. Number of vertices
2. Number of edges
3. Graph structure

Step 2: Initialize

Create two empty partitions:

$A = \{\}$

$B = \{\}$

Step 3: Process Vertices

For each vertex:

1. Count neighbors already present in partition A.

2. Count neighbors already present in partition B.
3. Place the vertex in the partition that creates more crossing edges.

Step 4: Continue Assignment

Repeat until all vertices are assigned to one of the two partitions.

Step 5: Compute Cut

Count all edges whose endpoints belong to different partitions.

Step 6: Output

Display:

- a. Partition A
- b. Partition B
- c. Crossing edges
- d. Approximate cut size

5. Approximation Strategy

The algorithm follows a greedy approximation approach.

For each vertex:

If $\text{edgesToA} > \text{edgesToB}$

Place vertex in B

Else

Place vertex in A

This decision attempts to maximize the number of crossing edges created at every step.

The approach guarantees a solution that is at least half as good as the optimal solution in many cases.

6. Example

Input Graph: Vertices = $\{1,2,3,4\}$

Edges:

(1,2)

(1,3)

(2,4)

(3,4)

Graph Representation

1 ----- 2

| |

| |

3 ----- 4

Partition Generated

A = $\{1,4\}$

B = $\{2,3\}$

Crossing Edges

(1,2)

(1,3)

(2,4)

(3,4)

Output

Partition A = $\{1,4\}$

Partition B = {2,3}

Approximate Cut Size = 4

This is the optimal cut for the given graph.

```
1 Q91 N-Queens Problem with Symmetry Elimination.py 3 Q55 Combination Generation with Sum Constraint.py 3 Q67 Approximation Maximum Cut Problem.py 3 X
```

```
C:\> Users > lenovo > OneDrive > Documents > 3 Q67 Approximation Maximum Cut Problem.py > ...
53     edges.append((u, v))
54
55 A, B, cut_edges = approximate_max_cut(vertices, edges)
56
57 print("\nPartition A =", A)
58 print("Partition B =", B)
59
60 print("\nCut Edges:")
61
62 for edge in cut_edges:
```

```
lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher '49438' '--' 'c:\Users\lenovo\OneDrive\Documents\3 Q55 Combination Generation with Sum Constraint.py'
PS C:\Users\lenovo\OneDrive\Documents> c:: cd 'c:\Users\lenovo\OneDrive\Documents'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '60930' '--' 'c:\Users\lenovo\OneDrive\Documents\3 Q67 Approximation Maximum Cut Problem.py'
Enter number of vertices: 4
Enter number of edges: 4
Enter edges (u v):
1 2
1 3
2 4
3 4

Partition A = {1, 4}
Partition B = {2, 3}

Cut Edges:
(1, 2)
(1, 3)
(2, 4)
(3, 4)

Approximate Cut Size = 4
PS C:\Users\lenovo\OneDrive\Documents>
```

7. Correctness of the Algorithm

The algorithm is correct because:

1. Every vertex is assigned to exactly one partition.
2. No vertex belongs to both partitions.
3. Every edge is evaluated correctly.
4. The cut size is computed by counting edges connecting different partitions.
5. The greedy assignment increases the likelihood of maximizing crossing edges.

Although the algorithm does not always produce the exact maximum cut, it guarantees a valid cut and provides a good approximation.

8. Complexity Analysis

Time Complexity

Building adjacency list: $O(E)$

Processing vertices: $O(V + E)$

Computing cut size: $O(E)$

Overall: $O(V + E)$

where:

- V = Number of vertices
- E = Number of edges

Space Complexity

Adjacency List: $O(V + E)$

Partitions: $O(V)$

Overall: $O(V + E)$

9. Advantages

1. Fast and efficient.
2. Suitable for large graphs.
3. Much simpler than exact algorithms.
4. Produces near-optimal solutions.
5. Requires relatively low memory.
6. Easy to implement and understand.

10. Limitations

1. Does not always find the optimal maximum cut.
2. Quality of solution may depend on vertex ordering.

3. Approximation ratio may vary for different graph structures.
4. Exact algorithms provide better results but require significantly more computation.

11. Applications

Network Design

Used to partition communication networks efficiently.

VLSI Circuit Design

Helps minimize connections between circuit modules.

Social Network Analysis

Detects communities and groups within networks.

Parallel Computing

Divides tasks among processors while minimizing communication overhead.

Data Clustering

Used in graph-based clustering techniques.

Machine Learning

Applied in optimization and graph partitioning problems.

Image Segmentation

Separates image regions using graph cuts.

12. Result

The Approximation Maximum Cut Problem was successfully solved using a greedy approximation algorithm. The algorithm partitioned graph vertices into two sets and maximized

the number of crossing edges while maintaining low computational complexity. The experiment demonstrated that approximation techniques can efficiently solve NP-Hard problems and provide high-quality solutions within practical execution time. The obtained results validate the effectiveness of approximation algorithms for graph partitioning and optimization tasks.